

3. GRADIENT BASED OPTIMIZATION

Optimum seeking is often necessary in engineering. The topics of the lecture:

- ✓ Introduction to Optimization problems
 - Definitions
 - The analytic conditions of the optimum
 - Example: Optimum problem in control theory
- ✓ Optimum seeking methods (unconstrained, finite dimensional)
 - Optimum search in one scalar variable
 - Gradient method
 - Conjugate gradient method
 - Newton-method
 - Gradient-like methods
 - Extension to constrained case (Use of Penalty functions)
- ✓ Parameter estimation
 - Batch mode
 - On-line mode (recursive form)
 - Nonlinear parameter estimation
(using local linearization and linear parameter estimation)

Important applications: Optimal function approximator e.g:

- Neural networks (NN),
- Fuzzy systems (FS) [pl. Sugeno Systems]

3.1. Analytic conditions of constrained optimization

It is known from preliminary studies that the necessary conditions of local minimum (x^*) of an unconstrained function $f(x)$, $x = (x_1, \dots, x_n)$ is

- $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ positive semi definite.

In the sequel, we are looking for the necessary conditions for (local) optimum that satisfies certain constraints.

Problem statement (Optimization problem): Searching point x^* which is the solution to the following optimization problem:

$$\begin{aligned} x^* &= \min_{x \in R^n} f(x) \\ c_i(x) &= 0, \quad i \in E && \text{(equality constraints)} \\ c_i(x) &\geq 0, \quad i \in I && \text{(inequality constraints)} \end{aligned}$$

where the objective function f and c_i are smooth, real-valued functions on a subset of R^n and I, E are two finite sets of indices.

Definition [Feasible set]. Set Ω is called feasible set if the set of points x satisfy the constraint; that is

$$\Omega = \{x \mid c_i(x) = 0, i \in E; \quad c_i(x) \geq 0, i \in I\}$$

The points of feasible set are called feasible points.

Compact form of constrained optimization problem: $x^* = \min_{x \in \Omega} f(x)$

Definition [Active set]. The active set $A(x)$ at any feasible x consists of the equality constraint indices from E together with the indices of the inequality constraint i for which $c_i(x) = 0$ i.e

$$A(x) = E \cup \{i \in I \mid c_i(x) = 0\}$$

Definition [LICQ condition]. Given the point x and the active set $A(x)$. We say that the Linear Independence Constraint Qualification (LICQ) holds, if the set of active constraint gradients $\{\nabla c_i(x), i \in A(x)\}$ is linearly independent.

Definíció [Lagrange function]

$$L(x, \lambda) = f(x) - \sum_{i \in E \cup I} \lambda_i c_i(x)$$

Now, we can state the first order (necessary) conditions for optimality. They are called ‘first order’ because they are concerned with properties of the gradients (first-derivative vectors) of the objective and constraint functions. These conditions are the foundations for many optimization algorithms.

Theorem [First order necessary conditions for optimality].

Suppose that

- x^* is a local solution of the optimization problem
- functions f and c_i are continuously differentiable
- LICQ condition holds at x^* .

Then there is a Lagrange multiplier vector λ^* with components λ_i^* , $i \in E \cup I$ such that the following conditions are satisfied at (x^*, λ^*) :

$$\nabla_x L(x^*, \lambda^*) = 0$$

$$c_i(x^*) = 0, \quad \forall i \in E$$

$$c_i(x^*) \geq 0, \quad \forall i \in I$$

$$\lambda_i^* \geq 0, \quad \forall i \in I$$

$$\lambda_i^* c_i(x^*) = 0, \quad \forall i \in E \cup I$$

Remark. These conditions are often known as the **Karush-Kuhn-Tucker (KKT)** conditions.

Remark. The last equality conditions are complementary conditions, i.e. they imply that either constraint i is active or $\lambda_i^* = 0$, or possibly both.

Remark. The Lagrange multipliers corresponding to inactive inequality constraints are zero so one can write

$$0 = \nabla_x L(x^*, \lambda^*) = \nabla f(x^*) - \sum_{i \in A(x^*)} \lambda_i^* \nabla c_i(x^*)$$

Remark. If the optimization problem is convex then KKT are *sufficient* conditions for *global* minimum.

Example [LQ optimal problem in control engineering]

$$F_0(x, u) = \frac{1}{2} \sum_{k=0}^N \langle Q_k x_k, x_k \rangle + \frac{1}{2} \sum_{k=0}^N \langle R_k u_k, u_k \rangle \rightarrow \min, \quad Q_k \geq 0, \quad R_k > 0,$$

Constraint set: $x_{k+1} = A_k x_k + B_k u_k, \quad k = 0, \dots, N, \quad x_0 = a$

The solution is searched for $x = \{x_k\}_{k=0}^N, \quad u = \{u_k\}_{k=0}^N$, which is a finite dimensional Euclidean space.

Since the objective function is convex and the constraint set is given by linear equalities hence the necessary and sufficient condition of global optimality is the Lagrange-multiplicator rule (KKT without inequality constraints)

3.2. Optimum seeking methods (unconstrained, finite dimensional)

- Often used in control engineering, modeling, signal processing
- we are involved only in minimum search
- If there are constraints, a penalty function is used. (Penalty function has a high value if any cpnstrained violated).

3.2.1. Optimum search in one scalar variable

The location of the minimum from a point x_k in direction d_k can be approximated by

$$\lambda_k \approx \arg \min \{ f(x_k + \lambda d_k) : \lambda \geq 0 \}$$

Two principles to find λ_k :

➤ **Cauchy-principle:** $\lambda_k \approx \arg \min \{ f(x_k + \lambda d_k) : \lambda \geq 0 \}$

➤ **Goldstein-principle:** Let q_1, q_2 , $0 < q_1 < q_2 < 1$. If

$$\lambda q_2 < f'(x_k), d_k > \leq f(x_k + \lambda d_k) - f(x_k) \leq \lambda q_1 < f'(x_k), d_k >$$

(G)

holds for $\lambda = 1$ then $\lambda_k := 1$, otherwise

a $\lambda_k > 0$ is to be found, where $\lambda = \lambda_k$ satisfies (G)

Remark. Expression $< f'(x_k), d_k >$ is the slope of function in direction d_k . Since d_k is chosen in negative gradient direction, this expression is negative in the optimum direction.

Remark. Value $\lambda < f'(x_k), d_k >$ **is the change of function in direction** d_k with step size λ .

Remark. q_2 is larger than q_1 which ensures that function must decrease in the next step ($\lambda_k d_k$) to satisfy (G).

More exactly:

$$\varphi(\lambda) := f(x_k + \lambda d_k) - f(x_k) \Rightarrow \varphi(0) = 0,$$

$$\varphi'(0) = < f'(x_k), d_k > \Rightarrow \varphi'(0) < 0,$$

$$\underline{\varphi}(\lambda) := \varphi(\lambda) - \lambda q_1 \varphi'(0),$$

$$\overline{\varphi}(\lambda) := \varphi(\lambda) - \lambda q_2 \varphi'(0).$$

[Goldstein algorithm]

$\overline{\varphi}(\lambda)$ is positive because with $\lambda q_2 \varphi'(0)$ one goes below the optimum value $\varphi(\lambda_{opt})$, i.e. $\varphi(\lambda_{opt})$ is estimated from below.

$\underline{\varphi}(\lambda)$ is negative and the change of $\lambda q_1 \varphi'(0)$ is not so negative as $\varphi(\lambda_{opt})$

Equivalent condition with (G): $\underline{\varphi}(\lambda_k) \leq 0, \quad \overline{\varphi}(\lambda_k) \geq 0$

3.2.1.1. Optimum search with Cauchy-principle

Assumption. Let $\varphi: R^1 \rightarrow R^1$ and $\varphi(\lambda)$ be convex and differentiable.

We are looking for the minimum of $\varphi(\lambda)$

Definition [Fibonacci number]. The solution of equation $F^2 + F - 1 = 0$ is the Fibonacci number $F = (\sqrt{5} - 1) / 2 \approx 0.618$.

The following algorithms find an interval (in λ) with length $\varepsilon > 0$ in which $\varphi(\lambda)$ takes the minimum. The closer we are to minimum the smaller the value $\varphi(\lambda)$ since $\varphi(\lambda) := f(x_k + \lambda d_k) - f(x_k)$ where first term takes the smallest value at the optimum (And the second term does not depend on λ).

Algorithm [Cauchy Principle, Fibonacci-search, Golden section].

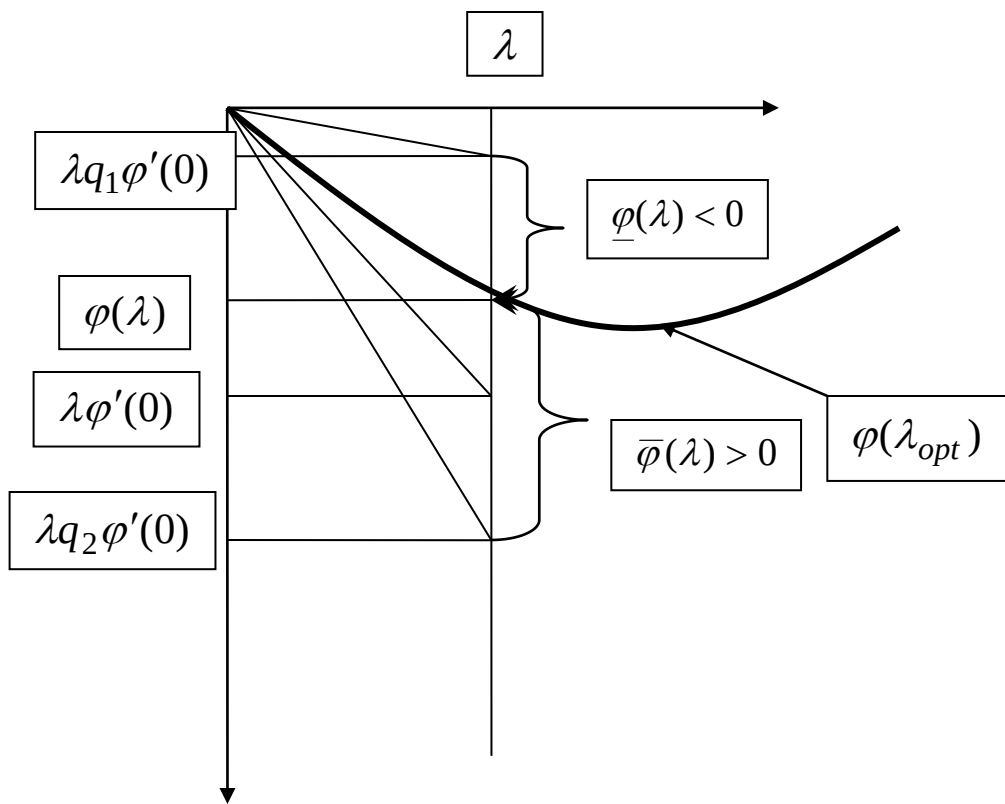
1. Determine an interval $[a_0, b_0]$, which contains λ_k , the place of the minimum. Let $i := 0$
2. If $b_i - a_i \leq \varepsilon$, then stop and the place of minimum satisfies $\lambda_k \in [a_i, b_i]$.
3. Let $v_i := b_i - F(b_i - a_i)$, $w_i := a_i + F(b_i - a_i)$ *(Partition with Fibonacci number)*
4. If $\varphi(v_i) < \varphi(w_i)$, then let $a_{i+1} := a_i$, $b_{i+1} := w_i$, $i := i + 1$ $\mapsto$ Jump to Step 2
5. If $\varphi(v_i) \geq \varphi(w_i)$, then let $a_{i+1} := v_i$, $b_{i+1} := b_i$, $i := i + 1$ $\mapsto$ Jump to Step 2

The algorithm approaches $\varphi(\lambda)$ from both side ($\lambda = v_i$ or $\lambda = w_i$). Sooner or later the optimum is limited to an interval with length ε .

3.2.1.2. Optimum search with Goldstein-principle

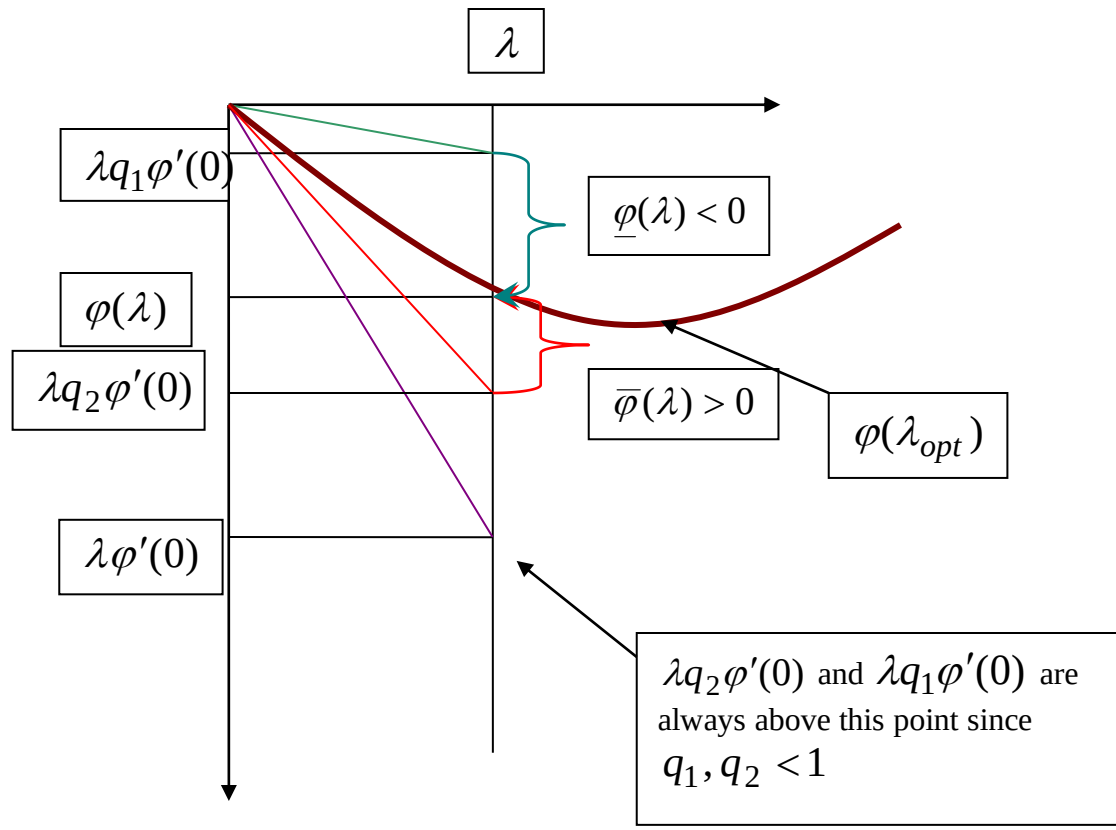
Algorithm [Goldstein-principle].

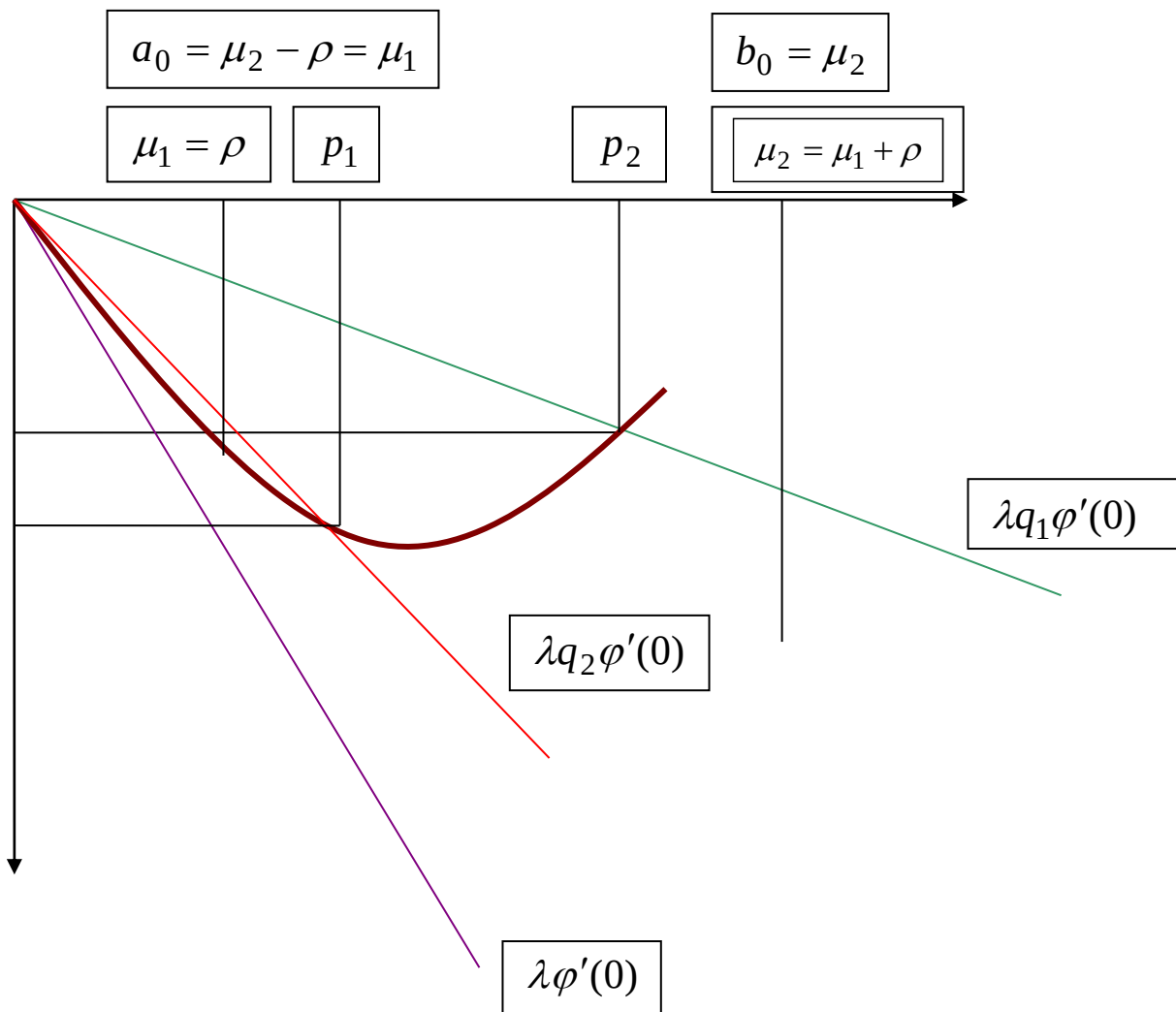
1. $\mu := \rho > 0$
2. Compute $\bar{\varphi}(\mu)$ számítása (earlier).
3. If $\bar{\varphi}(\mu) \approx 0$, then let $\lambda_k := \mu$, Stop.
If $\bar{\varphi}(\mu) < 0$, then let $\mu := \mu + \rho \mapsto$ Jump to Step 2
4. Compute $\underline{\varphi}(\mu)$ (Its value is positive see).
5. If $\underline{\varphi}(\mu) \leq 0$, then let $\lambda_k := \mu$, Stop
If $\underline{\varphi}(\mu) > 0$, then let $a_0 := \mu - \rho$, $b_0 := \mu$, $i := 0$
6. Let $v_i = \frac{1}{2}(a_i + b_i)$ and compute $\underline{\varphi}(v_i)$, $\bar{\varphi}(v_i)$.
7. If $\underline{\varphi}(v_i) \leq 0$ and $\bar{\varphi}(v_i) \geq 0$, then $\lambda_k := v_i$, Stop.
If $\underline{\varphi}(v_i) > 0$, then $a_{i+1} := a_i$, $b_{i+1} := v_i$, $i := i + 1 \mapsto$ Jump to Step 6
If $\bar{\varphi}(v_i) < 0$, then $a_{i+1} := v_i$, $b_{i+1} := b_i$, $i := i + 1 \mapsto$ Jump to Step 6



[\[algorithm\]](#)

Initial explanation





3.2.2. *Gradiens módszer*

- The gradient method is the simplest optimum seeking method in the case when no constraints are present.
- If $g_k := f'(x_k)$ denotes the gradient vector, then the decreasing direction is the negative gradient $d_k := -g_k$
- The new estimation of the place of the minimum can be found by optimum search along the direction d_k

based on

- Cauchy-principle
- Goldstein-principle
- With fix step size λ_0 where the new approximation of the minimum:

$$x_{k+1} := x_k - \lambda_0 \frac{g_k}{\|g_k\|}$$

3.2.3. Conjugate gradient method

Introduction.

Let us consider a convex and quadratic real-valued objective function:

$$f(x) = \frac{1}{2} \langle Ax, x \rangle + \langle b, x \rangle + c, \quad A > 0 \quad (\text{symmetrical and positive definite})$$

$$g := f'(x) = Ax + b \quad (\text{gradient})$$

$$f''(x) = A \quad (\text{Hessian matrix})$$

At the location x_0 of the minimum: $f'(x_0) = Ax_0 + b = 0$

One could obtain x_0 from this equation but we would like to extend the method for more general functions or for such cases, as the form $f''(x)$ of the Hessian matrix is unknown or it could be only hardly computed.

Therefore, conjugate gradient method

- will need only the gradient
- finds the optimum in $n = \dim x$ steps for quadratic functions.

Definition [Conjugate directions] We say that $\{d_0, d_1, \dots, d_k\}$ are conjugate directions relative to A , if $\langle d_k, Ad_i \rangle = 0$ and $\langle g_k, g_i \rangle = 0$ for every $i < k$

Algorithm [Conjugate gradient algorithm for quadratic functions]

1. Let x_0 be the initial point for :

$$g_0 := f'(x_0) = Ax_0 + b$$
$$d_0 = -g_0$$

2. Suppose, we already know
and the following hold:

$$x_k, g_k = f'(x_k) = Ax_k + b, d_k,$$
$$\forall i < k, \langle d_k, Ad_i \rangle = 0, \langle g_k, g_i \rangle = 0$$

Let x_{k+1} the place of minimum in the direction of d_k , and let us choose the following values:

$$\lambda_k := -\frac{\langle d_k, g_k \rangle}{\langle d_k, Ad_k \rangle}, \quad \text{[[Explanation 1](#)]}$$

$$x_{k+1} := x_k + \lambda_k d_k,$$

$$g_{k+1} := f'(x_{k+1}) = Ax_{k+1} + b,$$

$$\beta_k := \frac{\langle g_{k+1}, Ad_k \rangle}{\langle d_k, Ad_k \rangle}, \quad \text{[[Explanation 2](#)]}$$

$$d_{k+1} := -g_{k+1} + \beta_k d_k$$

Repeat step 2 until $k = n$ is reached, then x_n is the place of the minimum.

Explanations:

1. Let $\varphi(\lambda) := f(x_k + \lambda d_k)$, then $\varphi'(\lambda) = 0$ at the place of minimum along d_k , hence

$$\varphi'(\lambda) = \langle f'(x_k + \lambda d_k), d_k \rangle = \langle A(x_k + \lambda d_k) + b, d_k \rangle = 0, \quad \text{and by using}$$
$$g_k = Ax_k + b$$

it follows [expression](#) $\lambda_k := - \langle d_k, g_k \rangle / \langle d_k, Ad_k \rangle$

2. Let us choose $d_{k+1} := -g_{k+1} + \beta_k d_k$, then, adding d_{k+1} to the conjugate directions, $\langle d_{k+1}, Ad_k \rangle = \langle -g_{k+1} + \beta_k d_k, Ad_k \rangle = 0$ should be satisfied, from which it follows $\beta_k = \langle g_{k+1}, Ad_k \rangle / \langle d_k, Ad_k \rangle$ [see expression](#).

Lemma: $\{d_0, d_1, \dots, d_{k+1}\}$ are conjugate directions relative to A .

Proof:

i) First we show that $\forall i < k + 1, \langle g_i, g_{k+1} \rangle = 0$.

By definition of g_{k+1} :

$$g_{k+1} = Ax_{k+1} + b = A(x_k + \lambda_k d_k) + b = g_k + \lambda_k Ad_k$$

$$\langle g_i, g_{k+1} \rangle = \langle g_i, g_k \rangle + \lambda_k \langle g_i, Ad_k \rangle.$$

If $i < k$, then by construction ([reordering the last expression of the algorithm](#)) and the definition of conjugate directions:

$$g_i = -d_i + \beta_{i-1}d_{i-1} \Rightarrow \langle g_i, g_{k+1} \rangle = \langle g_i, g_k \rangle + \lambda_k \langle -d_i + \beta_{i-1}d_{i-1}, Ad_k \rangle = 0$$

since each scalar product is zero at the right hand side (using $\langle d_k, Ad_i \rangle = 0$ and $\langle g_k, g_i \rangle = 0$).

If $i = k$, then the [value](#) of λ_k and g_k (expressing from the last [expression](#) of the algorithm) it yields

$$\langle g_k, g_{k+1} \rangle = \langle g_k, g_k \rangle + \lambda_k \langle g_k, Ad_k \rangle = \langle g_k + d_k, g_k \rangle = \langle \beta_{k-1}d_{k-1}, g_k \rangle = 0,$$

since (by recursion) $\beta_{k-1}d_{k-1} \in \text{Span}\{g_0, \dots, g_{k-1}\}$ and we have seen that $\langle g_i, g_k \rangle = 0$ for every $i < k + 1$.

ii) After this we shall show that $\langle d_{k+1}, Ad_i \rangle = 0$ for every $\forall i < k + 1$.

The case $i = k$ is surely satisfied because of the choice of β_k [\[explanation 2\]](#).

$$\text{If } i < k, \text{ then } \langle d_{k+1}, Ad_i \rangle = \langle -g_{k+1} + \beta_k d_k, Ad_i \rangle = -\langle g_{k+1}, Ad_i \rangle \quad (\text{T})$$

by the construction, and by the choice of x_{i+1}

$$g_{i+1} = Ax_{i+1} + b = A(x_i + \lambda_i d_i) + b = g_i + \lambda_i Ad_i \Rightarrow Ad_i = \frac{1}{\lambda_i} (g_{i+1} - g_i), \quad (\text{S})$$

from which it follows by i) on the base of (T),(S) : $\langle d_{k+1}, Ad_i \rangle \stackrel{(T)}{=} \langle g_{k+1}, Ad_i \rangle \stackrel{(S)}{=} \frac{1}{\lambda_i} \langle g_{k+1}, g_{i+1} - g_i \rangle = 0$

Remark: assuming convex and quadratic function we were able to derive that $\langle g_k, g_i \rangle = 0$ for every $i < k$.

Lemma: If $f(x) = \frac{1}{2} \langle Ax, x \rangle + \langle b, x \rangle + c$, $A \succ 0$, then the algorithm finds the minimum within at most $n = \dim x$ steps.

Proof:

The necessary and sufficient condition of the minimum needs to find such x_k for which

$$f'(x_k) = Ax_k + b =: g_k = 0.$$

If during the algorithm $g_k = 0$ occurs for $k < n$ then we have already found the minimum. If this is not the case, then by the construction $\{g_0, \dots, g_{n-1}\}$ is an orthonormal system, $\text{Span}\{g_0, \dots, g_{n-1}\} = R^n$, and $g_i \perp g_n$, $\forall i < n$, hence $g_n \perp R^n$ and $g_n = 0$, which is the necessary and sufficient condition of the global minimum

Algorithm [Conjugate gradient algorithm for general function]

1. Initialization. Let x_0 be the starting point and $g_0 := f'(x_0)$, $d_0 = -g_0$, $k = 0$.
2. Suppose that we already know x_k , $g_k = f'(x_k)$ és d_k .

Find the minimum of the function $\varphi(\lambda) = f(x_k + \lambda d_k)$

(Cauchy-rprinciple, Goldstein principle, fix steps ec), and let λ_k be the place of minimum..

Determine the new search direction by the following rule:

$$x_{k+1} := x_k + \lambda_k d_k, \quad g_{k+1} := f'(x_{k+1})$$

$$\beta_k := \frac{\langle g_{k+1}, g_{k+1} - g_k \rangle}{\langle d_k, g_{k+1} - g_k \rangle}, \quad (\text{substituting [expression \[S\]](#) into the } Ad_k \text{ of the previous algorithm.})$$

$$d_{k+1} := -g_{k+1} + \beta_k d_k$$

Repeat Step 2 until $k = n$ if $\|g_k\| > 0$.

After $k = n$ reinitialization: $x_0 := x_n$

If $\|g_k\| \approx 0$, then x_k is accepted as the location of the minimum.

Megjegyzés. If $f(x)$ is non-quadratic then the method is only approximating, and generally the minimum can not be reached within n steps.

3.2.4. Newton-method

Let $f(x)$ be a convex and quadratic function written in the form

$$\begin{aligned} f(x) &= \frac{1}{2} A(x - x_0, x - x_0), \quad A > 0, \\ g &:= f'(x) = A(x - x_0), \\ f''(x) &= A \end{aligned}$$

where x_0 is the place of global minimum. It is evident that for any x yields

$$x_0 = x - A^{-1}\{A(x - x_0)\}$$

which can be written in the Newtonian form:

$$x_0 := x - [f''(x)]^{-1} f'(x)$$

(N)

The minimum of any convex and quadratic function can be found in one step for any starting point. However the method needs not only the gradient vector but also the Hessian matrix. If the function is non-quadratic the method is only approximating:

Approximation of Newton-method: $x_{k+1} := x_k - [f''(x_k)]^{-1} f'(x_k)$ (Approximation if the function is not quadratic)

Quasi Newton-method: $x_{k+1} := x_k - [H(x_k)]^{-1} f'(x_k)$ ($f''(x) \approx H(x)$ approximation)

Levenberg-Marquadt method: $H(x_k) := H(x_k) + \delta I$ (δ is a small number)

3.2.5 Gradient-like methods

Problem: During optimum search it is usually expected that $f(x_{k+1}) < f(x_k)$ if decreasing directions are used. Unfortunately this condition can usually not assure that in the x^* accumulation (limit) points of the sequence $\{x_k\}$ the gradient becomes zero which is the necessary condition of the minimum.
(Example: search direction is tangent to the isocost curve)

To this decreasing direction with appropriate slope should be assured

Definition [F-function]. The function $\rho : [0, \infty) \rightarrow [0, \infty)$ is called F -function (forcing function), if for each $\{t_k\} \subset [0, \infty)$ sequence

$$\lim_{k \rightarrow \infty} \rho(t_k) = 0 \Rightarrow \lim_{k \rightarrow \infty} t_k = 0$$

In other words, F-function converge to zero only if the argument converge to zero as well.

In the next we assume the [V]-condition for the minimum problem $\min_{x \in R^n} f(x)$.

[V]-Condition: Starting point x_0 for the optimum search satisfies that $C(x_0) := \{x \in R^n : f(x) \leq f(x_0)\}$ is a compact set.

Algorithm [gradient-like procedures].

1. Choose an initial $x_0 \in R^n$, $k := 0$.
2. If $f'(x_k) \approx 0$, then let $x^* := x_k$ be the approximation of the place of minimum, stop
3. Compute a $d_k \neq 0$ direction vector, such that for every $x_k \in C(x_0)$ and the fixed $\rho(\cdot)$ F-function the following inequality is satisfied

$$\sigma_k := \frac{\langle f'(x_k), d_k \rangle}{\|d_k\|} \leq -\rho(\|f'(x_k)\|)$$

(Z)

(i.e we are approaching to a zero gradient but we have freedom to choose direction $d_k \neq 0$).

The gradient method satisfies condition (Z) with $\rho(t) = t$

4. Determine λ_k either by Cauchy-principle or by Goldstein-principle.
5. Let $x_{k+1} := x_k + \lambda_k d_k$, $k := k + 1$ and jump to step 2.

3.2.5.1. Davidson/Fletcher/Powell method

Some extensions are applied.

In Step 1 (Initialization): $H_0 > 0$ (Choose positive definite matrix).

In Step 3: $d_k := -H_k f'(x_k)$

In Step 4: determination of λ_k by Cauchy-principle.

In Step 5:

$$q_k := f'(x_{k+1}) - f'(x_k), \quad r_k := x_{k+1} - x_k,$$
$$H_{k+1} := H_k + \frac{r_k r_k^T}{\langle r_k, q_k \rangle} - \frac{(H_k q_k)(H_k q_k)^T}{\langle q_k, H_k q_k \rangle}.$$

Remarks:

- $H_k > 0$ is guaranteed by the algorithm.
- If $f''(x)$ fulfils some smoothness and boundedness conditions then the Davidon/Fletcher/Powell method is a gradient-like method
- $\rho(t)$ can be determined from the bounds
- The convergence is stronger than linear.

3.2.5.2. Fletcher/Reeves and the Polak/Ribiere methods

In Step 1 (Initialization): $\beta_0 := 0$

In Step 3: Search direction: $d_k := -f'(x_k) + \beta_{k-1}d_{k-1}$

In Step 4: determination of λ_k by Cauchy-principle

In Step 5: β_k is corrected by the following rules:

$$\beta_k := \frac{\|f'(x_{k+1})\|^2}{\|f'(x_k)\|^2} \quad (\text{Fletcher/Reeves})$$

$$\beta_k := \frac{\langle f'(x_{k+1}) - f'(x_k), f'(x_{k+1}) \rangle}{\|f'(x_k)\|^2} \quad (\text{Polak/Ribiere})$$

Remarks:

- In the case of convex and quadratic function the search directions d_k are conjugate directions, and the two methods are identical.
- If the function is non-quadratic and $n = \dim x$, then it is useful to re-initialize the procedure after every n -th step: $\beta_k := 0$, if $k = 0 \pmod{n}$

3.3. Linear parameter estimation

Important tool to numerous engineering problem.

Goal: Matching the parameters to noisy measurements (observations)

$$y(i) = \varphi^T(i)\mathcal{G} + \varepsilon(i), \quad y \in R^m, \quad \mathcal{G} \in R^p, \quad \varphi^T \quad \text{is a } m \times p \text{ matrix}$$

$y(i), \varphi^T(i)$ output and other observations/measurements from the past (m is the number of output)

$\mathcal{G}$ Unknown parameter vector (the goal is to obtain it)

$\varepsilon(i)$ Unknown noise (and $\varepsilon(i) = y(i) - \varphi^T(i)\mathcal{G}$ is the equation error or residual)

$$i = 1, 2, \dots, t$$

in case of $y \in R^1$ (single variable), $\varphi^T(i)$ is a row vector.

To be examined:

1. The number of observations (measurements) is fix: $t = N$ (off-line, batch method)
2. Data is collected in real time (on-line, recursive parameter estimation)

3.3.1. Linear parameter estimation in batch mode

3.3.1.1. Ordinary least squares (LS) method

Idea: minimizing the sum of the squares of the equation errors in time interval

$$V(\mathcal{G}, t) = \frac{1}{2} \sum_{i=1}^t \varepsilon^2(i) = \frac{1}{2} \sum_{i=1}^t \|y(i) - \varphi^T(i) \mathcal{G}\|^2 = \frac{1}{2} \|Y - \Phi \mathcal{G}\|^2, \quad Y = \begin{pmatrix} y(1) \\ \vdots \\ y(t) \end{pmatrix}, \quad \Phi = \begin{bmatrix} \varphi^T(1) \\ \vdots \\ \varphi^T(t) \end{bmatrix}, \quad E = \begin{pmatrix} \varepsilon(1) \\ \vdots \\ \varepsilon(t) \end{pmatrix},$$

$$\mathcal{G} = \arg \min_{\mathcal{G} \in \mathbb{R}^p} V(\mathcal{G}, t).$$

$V(\mathcal{G}, t)$ is convex, quadratic in $\mathcal{G} \rightarrow$ the necessary and sufficient condition of optimality is that the derivative by $\mathcal{G}$ is zero: $V'(\mathcal{G}, t) = 0$

$$\begin{aligned} V(T, \mathcal{G}) &= \|Y - \Phi \mathcal{G}\|^2 = \langle Y - \Phi \mathcal{G}, Y - \Phi \mathcal{G} \rangle = \langle Y, Y \rangle - 2 \langle Y, \Phi \mathcal{G} \rangle + \langle \Phi \mathcal{G}, \Phi \mathcal{G} \rangle = \\ &= \langle Y, Y \rangle - 2 \langle \Phi^T Y, \mathcal{G} \rangle + \langle \Phi^T \Phi \mathcal{G}, \mathcal{G} \rangle, \end{aligned}$$

Differentiating:

$$V'(\mathcal{G}, t) = \frac{dV}{d\mathcal{G}} = -\Phi^T Y + \Phi^T \Phi \mathcal{G} = 0$$

The optimal LS solution is $\hat{\mathcal{G}}$, which is obtained by:

$$\begin{aligned} \Phi^T \Phi \hat{\mathcal{G}} &= \Phi^T Y, \\ \hat{\mathcal{G}} &= (\Phi^T \Phi)^{-1} \Phi^T Y \end{aligned}$$

normal equation (numerically better conditioned than the next one)
(the observations should be persistently exciting in order to determine the parameters, i.e. $\Phi^T \Phi$ should be invertible)

3.3.1.2. Weighted least squares (LS) method

Difference from ordinary LS: equation error is weighted by positive definit matrix $W > 0$

$$\begin{aligned} V(\mathcal{G}, t) &= \frac{1}{2} \langle WE, E \rangle = \frac{1}{2} \langle W(Y - \Phi \mathcal{G}), Y - \Phi \mathcal{G} \rangle = \frac{1}{2} \{ \langle WY, Y \rangle - \langle W\Phi \mathcal{G}, Y \rangle - \langle WY, \Phi \mathcal{G} \rangle + \langle W\Phi \mathcal{G}, \Phi \mathcal{G} \rangle \} = \\ &= \frac{1}{2} \{ \langle WY, Y \rangle - 2 \langle \Phi^T WY, \mathcal{G} \rangle + \langle \Phi^T W\Phi \mathcal{G}, \mathcal{G} \rangle \}. \end{aligned}$$

Then

$$V'(\mathcal{G}, t) = \frac{dV}{d\mathcal{G}} = -\Phi^T WY + \Phi^T W\Phi \mathcal{G} = 0$$

and

$$\begin{aligned} \Phi^T W\Phi \hat{\mathcal{G}} &= \Phi^T WY \\ \hat{\mathcal{G}} &= (\Phi^T W\Phi)^{-1} \Phi^T WY \end{aligned}$$

- W increases considerably the computational time. if W is block diagonal then
 - choice $\sqrt{w_{ii}}y(i)$, $\sqrt{w_{ii}}\varphi^T(i)$ simplifies the computations.
 - $Z^T = \Phi^T W$, i.e. instead of $\varphi(i)$, one can use $\varphi(i)w_{ii}$ and $\hat{\mathcal{G}} = (Z^T \Phi)^{-1} Z^T Y$ which simplifies computations as well. (relation to IV method)

3.3.1.3..Statistical properties:

1. If $\mathcal{G}$ and Φ deterministic and mean value of $E = Y - \Phi\mathcal{G}$ is zero $\rightarrow$ LS estimation is unbiased ($\hat{\mathcal{G}} = \mathcal{G}_0$)
2. In stochastic case, if E and Φ are independent, furthermore the mean value of E is zero $\rightarrow$ LS estimation remains unbiased
+ if error E hiba uncorrelalated and has uniform variance ($\text{cov}[E, E] = \sigma^2 I$) $\rightarrow$ kovariancia minimális
(in the the class of unbiased linear estimations)
3. If $\text{cov}[E, E] = \sigma^2 I$ is not satisfied $\rightarrow$ the unbiased estimation, which has minimal covariance,
is a WLS estimation, $W = (\text{cov}[E, E])^{-1}$
(This is the **Markov estimation**)

3.1.1.3. Main Numerical method for LS estimation: SVD

SVD (Singular Value Decomposition – Szinguláris érték felbontás)

Let Φ with size $M \times p := (Nm) \times p$, then

$$\Phi = U\Sigma V^T \quad U = U_{M \times M}, V = V_{p \times p}, UU^T = I, VV^T = I,$$

$$\Sigma = \begin{bmatrix} \Sigma_1 \\ 0 \end{bmatrix}_{M \times p} \quad \text{and} \quad \Sigma_1 = \text{diag}(\sigma_i), \text{ where } \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$$

σ_i : singular value of Φ

λ_i : the eigenvalue of $\Phi^T \Phi$ where $\sigma_i = \sqrt{\lambda_i}$

SVD:

1. Householder transformation: yields Φ in bidiagonal form
2. QR algorithms (with sequence of iteration): yields diagonal form

U : the orthonormed basis vectors of the image space $\text{range}(\Phi)$

V : orthonormed basis vectors of the kernel space $\text{kernel}(\Phi)$

U and V are orthogonal, hence

$$\Phi^T \Phi V = V \Sigma^T U^T U \Sigma V^T V = V \Sigma^T \Sigma = V \Sigma_1^T \Sigma_1 = V \text{diag}(\sigma_i^2)$$

v_i : the i th column of matrix V , at the same time the eigenvector of $\Phi^T \Phi$ with eigenvalue $\lambda_i = \sigma_i^2$.

Introducing transformation U^T to Y and transformation V^T to $\mathcal{G}$:

$$Y^* = U^T Y, \quad \mathcal{G}^* = V^T \mathcal{G}, \quad Y^* = U^T Y = U^T \Phi \mathcal{G} = U^T U \Sigma V^T \mathcal{G} = \Sigma V^T \mathcal{G} = \Sigma \mathcal{G}^*.$$

The transformations leave unchanged the sum of the squares of the equation errors:

$$Y^* - \Sigma \mathcal{G}^* = U^T Y - \Sigma V^T \mathcal{G} = U^T (Y - U \Sigma V^T \mathcal{G}), \quad \|Y^* - \Sigma \mathcal{G}^*\|^2 = \langle U^T (Y - U \Sigma V^T \mathcal{G}), U^T (Y - U \Sigma V^T \mathcal{G}) \rangle = \|Y - \Phi \mathcal{G}\|^2$$

Hence instead of $\|Y - \Phi \mathcal{G}\|^2$, one can minimize $\|Y^* - \Sigma \mathcal{G}^*\|^2$

$$\hat{\mathcal{G}}^* = (\Sigma^T \Sigma)^{-1} \Sigma^T Y^* = (\Sigma_1^T \Sigma_1)^{-1} [\Sigma_1^T \quad 0^T] Y^* = [\text{diag}(\sigma_i^{-1}) \quad 0^T] Y^*,$$

$$\hat{\mathcal{G}}_i^* = Y_i^* / \sigma_i, \quad i = 1, 2, \dots, p. \quad (\text{S1})$$

$$\hat{\mathcal{G}} = (V^T)^{-1} \hat{\mathcal{G}}^* = V \hat{\mathcal{G}}^* = \sum_i \hat{\mathcal{G}}_i^* v_i \quad (\text{S2})$$

Remarks on SVD:

If the columns (regressors) of Φ are linearly dependent $\rightarrow$

σ_i becomes small $\rightarrow$

$(Y_i^* / \sigma_i)v_i$ becomes large in parameter vector $\hat{\mathcal{G}}$ (see formulae S1) $\rightarrow$

Large equation error unless v_i is masked out:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 \quad \text{and} \quad \sigma_{r+1} = \dots = \sigma_p = 0 \Rightarrow \Sigma = \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix}_{M \times p}$$

Hence

$$\hat{\mathcal{G}}_i^* = \begin{cases} Y_i^* / \sigma_i, & \text{if } \sigma_i > 0 \\ 0, & \text{otherwise} \end{cases}$$

Remark. If $\Phi = U\Sigma V^T$ and diagonal matrix Σ_r contains the nonzero singular values, then

$$\Phi = U\Sigma V^T, \quad \Sigma = \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \Rightarrow \Sigma^+ = \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & 0 \end{bmatrix}, \quad \Phi^+ = V\Sigma^+U^T,$$

$$\hat{\mathcal{G}} := \Phi^+ Y.$$

the above (masking out) SVD method is equivalent to the method in which optimal parameter estimation is determined by Moore-Penrose pseudoinverse.

3.3.2. Recursive linear parameter estimation

- the estimation process is continuous
- If the parameter vector $\mathcal{G}$ is not constant during the estimation $\rightarrow$ weights λ^{t-i} for progressive forgetting where λ is the forgetting factor ($\lambda = 1 \rightarrow$ no forgetting)

$$V(\mathcal{G}, t) = \frac{1}{2} \sum_{i=1}^t \lambda^{t-i} \|y(i) - \varphi^T(i) \mathcal{G}\|^2 = \frac{1}{2} \sum_{i=1}^t \langle \lambda^{t-i} [y(i) - \varphi^T(i) \mathcal{G}], y(i) - \varphi^T(i) \mathcal{G} \rangle.$$

$m = \dim y \rightarrow$ Recall: $\Lambda := W = \text{diag}(\lambda^{t-1} I_m, \dots, \lambda^1 I_m, \lambda^0 I_m)$

$\dim y = 1 \rightarrow \Lambda := \text{diag}(\lambda^{t-1}, \dots, \lambda, 1)$

$V(\mathcal{G}, t)$ is convex, quadratic hence the global optimum:

$$\begin{aligned} \Phi^T \Lambda \Phi \hat{\mathcal{G}}(t) &= \Phi^T \Lambda Y && \text{(normal equation)} \\ \hat{\mathcal{G}}(t) &= (\Phi^T \Lambda \Phi)^{-1} \Phi^T \Lambda Y && \text{(B1)} \end{aligned}$$

Our goal is a recursive formula. Introducing notation

$$P(t) := (\Phi^T \Lambda \Phi)^{-1} = \left[\sum_{i=1}^t \lambda^{t-i} \varphi(i) \varphi^T(i) \right]^{-1}$$

one can write

$$P(t) = \left[\sum_{i=1}^{t-1} \lambda \cdot \lambda^{t-1-i} \varphi(i) \varphi^T(i) + \varphi(t) \varphi^T(t) \right]^{-1} = \left[\lambda P^{-1}(t-1) + \varphi(t) \varphi^T(t) \right]^{-1} \quad \text{(R1)}$$

Applying matrix inverse lemma $(A + BCD)^{-1} = A^{-1} - A^{-1}B(C^{-1} + DA^{-1}B)^{-1}DA^{-1}$ with the choice

$A := \lambda P^{-1}(t-1)$, $B := \varphi(t)$, $C := I$, $D := \varphi^T(t)$

$$P(t) = \frac{1}{\lambda} P(t-1) - \frac{1}{\lambda} P(t-1)\varphi(t) \left[I + \varphi^T(t) \frac{1}{\lambda} P(t-1)\varphi(t) \right]^{-1} \varphi^T(t) \frac{1}{\lambda} P(t-1) \quad (\text{R}^*)$$

From which one can get with simple manipulations that

$$P(t) = \frac{1}{\lambda} \{ P(t-1) - P(t-1)\varphi(t) [\lambda I + \varphi^T(t) P(t-1)\varphi(t)]^{-1} \varphi^T(t) P(t-1) \}$$

It follows from (B1) that

$$\hat{\mathcal{G}}(t) = P(t) \left[\lambda \sum_{i=1}^{t-1} \lambda^{t-1-i} \varphi(i) y(i) + \varphi(t) y(t) \right] \quad (\text{R2})$$

It can be written with a shifted phase

$$\hat{\mathcal{G}}(t-1) = P(t-1) \sum_{i=1}^{t-1} \lambda^{t-1-i} \varphi(i) y(i) \Rightarrow \sum_{i=1}^{t-1} \lambda^{t-1-i} \varphi(i) y(i) = P^{-1}(t-1) \hat{\mathcal{G}}(t-1). \quad (\text{R3})$$

One can get from (R1) that

$$P^{-1}(t-1) = \frac{1}{\lambda} \left[P^{-1}(t) - \varphi(t) \varphi^T(t) \right] \quad (\text{R4})$$

Substituting (R3) and (R4) into (R2)

$$\hat{\mathcal{G}}(t) = P(t) \left\{ \lambda \frac{1}{\lambda} \left[P^{-1}(t) - \varphi(t) \varphi^T(t) \right] \hat{\mathcal{G}}(t-1) + \varphi(t) y(t) \right\}$$

$$\hat{\mathcal{G}}(t) = \hat{\mathcal{G}}(t-1) + P(t) \varphi(t) [y(t) - \varphi^T(t) \hat{\mathcal{G}}(t-1)].$$

If one defines notation $K(t) := P(t) \varphi(t)$ then using (R*):

$$K(t) = \frac{1}{\lambda} \{ P(t-1) - P(t-1)\varphi(t) [\lambda I + \varphi^T(t) P(t-1)\varphi(t)]^{-1} \varphi^T(t) P(t-1) \} \varphi(t)$$

$$= \frac{1}{\lambda} P(t-1) \varphi(t) \{ I - [\lambda I + \varphi^T(t) P(t-1)\varphi(t)]^{-1} \varphi^T(t) P(t-1) \varphi(t) \} \Rightarrow$$

$$K(t) = P(t-1) \varphi(t) [\lambda I + \varphi^T(t) P(t-1)\varphi(t)]^{-1}$$

Results

Algorithm [recursive parameter estimation] – The popular form in signal processing:

$$P(t) = \frac{1}{\lambda} \{P(t-1) - P(t-1)\varphi(t)[\lambda I + \varphi^T(t)P(t-1)\varphi(t)]^{-1}\varphi^T(t)P(t-1)\}$$
$$\hat{\mathcal{G}}(t) = \hat{\mathcal{G}}(t-1) + P(t)\varphi(t)[y(t) - \varphi^T(t)\hat{\mathcal{G}}(t-1)]$$

Algorithm [recursive parameter estimation] – The popular form in control engineering:

$$K(t) = P(t-1)\varphi(t)[\lambda I + \varphi^T(t)P(t-1)\varphi(t)]^{-1}$$
$$P(t) = [I - K(t)\varphi^T(t)]P(t-1) / \lambda$$
$$\hat{\mathcal{G}}(t) = \hat{\mathcal{G}}(t-1) + K(t)[y(t) - \varphi^T(t)\hat{\mathcal{G}}(t-1)]$$

(Previous estimation is corrected by a residual with gain $K(t)$)

Output prediction: $\hat{y}(t) = \varphi^T(t)\hat{\mathcal{G}}(t-1)$

Prediction error: $\varepsilon(t) = y(t) - \hat{y}(t)$

Initial values are needed, e.g: $\hat{\mathcal{G}}(0) := 0$, $P(0) := \sigma I$

3.4. Nonlinear parameter estimation by local linearization

Linear parameter estimation at time: $y(t) = \varphi^T(t) \mathcal{G}_0$

Generalization to nonlinear case: $y(t) = h(\mathcal{G}_0, t)$

Function h is known analytically but $\mathcal{G}_0$ is unknown (The goal is to find it!)

Let us assume that an approximation $\hat{\mathcal{G}}(t-1)$ is available. Then it is possible to determine the Taylor-series of function $h(\mathcal{G}, t)$ around $\hat{\mathcal{G}}(t-1)$ and to take the linear approximation of the nonlinear function:

$$h(\mathcal{G}_0, t) \approx h(\hat{\mathcal{G}}(t-1), t) + \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right|_{\hat{\mathcal{G}}(t-1)} (\mathcal{G}_0 - \hat{\mathcal{G}}(t-1)),$$
$$y(t) \approx h(\hat{\mathcal{G}}(t-1), t) - \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right|_{\hat{\mathcal{G}}(t-1)} \hat{\mathcal{G}}(t-1) + \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right|_{\hat{\mathcal{G}}(t-1)} \mathcal{G}_0$$

Introducing the notations

$y(t) := y(t) - h(\hat{\mathcal{G}}(t-1), t) + \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right _{\hat{\mathcal{G}}(t-1)} \hat{\mathcal{G}}(t-1)$ $\varphi^T(t) := \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right _{\hat{\mathcal{G}}(t-1)} = \frac{dh(\hat{\mathcal{G}}(t-1), t)}{d\mathcal{G}}$	(K1)
---	------

the nonlinear parameter estimation problem after local linearization can be transformed to the basic form $y(t) = \varphi^T(t) \mathcal{G}_0$ of the linear parameter estimation problems.

After local linearization the nonlinear parameter estimation problem can be traced back to recursive linear parameter estimation

$$\begin{aligned} y(t) - \varphi^T(t) \hat{\mathcal{G}}(t-1) &:= y(t) - h(\hat{\mathcal{G}}(t-1), t) + \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right|_{\hat{\mathcal{G}}(t-1)} \hat{\mathcal{G}}(t-1) - \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right|_{\hat{\mathcal{G}}(t-1)} \hat{\mathcal{G}}(t-1) = \\ &= y(t) - h(\hat{\mathcal{G}}(t-1), t). \end{aligned} \quad (\text{K2})$$

the residual can be computed immediately by the nonlinear model $h(\mathcal{G}, t)$, no need for correction step (K1)

Algorithm [recursive nonlinear parameter estimation (with local linearization)]

$$\varphi^T(t) := \left. \frac{dh(\mathcal{G}, t)}{d\mathcal{G}} \right|_{\hat{\mathcal{G}}(t-1)} \quad (\text{NP1})$$

$$P(t) = \frac{1}{\lambda} \{ P(t-1) - P(t-1) \varphi(t) [\lambda I + \varphi^T(t) P(t-1) \varphi(t)]^{-1} \varphi^T(t) P(t-1) \} \quad (\text{NP2})$$

$$\hat{\mathcal{G}}(t) = \hat{\mathcal{G}}(t-1) + P(t) \varphi(t) [y(t) - h(\hat{\mathcal{G}}(t-1), t)] \quad (\text{NP3})$$

$$t := t + 1. \quad (\text{NP4})$$

where (K2) is used in the formulae of linear parameter estimation and yields (NP3)

3.4 Optimization Toolbox

Complexity increases with the order of appearance in this list

Minimization:

Type	Notation	Function
Scalar minimization	$\min_x f(x)$ such that $x_1 < x < x_2$	fminbnd
Unconstrained minimization	$\min_x f(x)$	fminunc, fminsearch
Linear programming	$\min_x f^T x$ such that $A \cdot x \leq b, \quad Aeq \cdot x = beq, \quad l \leq x \leq u$	linprog
Quadratic programming	$\min_x \frac{1}{2} x^T H x + f^T x$ such that $A \cdot x \leq b, \quad Aeq \cdot x = beq, \quad l \leq x \leq u$	quadprog

Type	Notation	Function
Constrained minimization	$\min_x f(x) \text{ such that}$ $c(x) \leq 0, \text{ } ceq(x) = 0$ $A \cdot x \leq b, \text{ } Aeq \cdot x = beq, \text{ } l \leq x \leq u$	fmincon
Goal attainment	$\min_{x, \gamma} \gamma \text{ such that}$ $F(x) - w\gamma \leq \text{goal}$ $c(x) \leq 0, \text{ } ceq(x) = 0$ $A \cdot x \leq b, \text{ } Aeq \cdot x = beq, \text{ } l \leq x \leq u$	fgoalattain
Minimax	$\min_x \max_{\{F_i\}} \{F_i(x)\} \text{ such that}$ $c(x) \leq 0, \text{ } ceq(x) = 0$ $A \cdot x \leq b, \text{ } Aeq \cdot x = beq, \text{ } l \leq x \leq u$	fminimax

Semi-infinite minimization	$\min_x f(x) \text{ such that}$ $K(x, w) \leq 0 \text{ for all } w$ $c(x) \leq 0, \text{ } ceq(x) = 0$ $A \cdot x \leq b, \text{ } Aeq \cdot x = beq, \text{ } l \leq x \leq u$	fseminf
Binary integer programming	$\text{minimize } f' \cdot x$ such that $A \cdot x \leq b$ $Aeq \cdot x = beq$ $x \text{ binary}$	bintprog

Equation Solving

Type	Notation	Function
Linear equations	$C \cdot x = d$, n equations, n variables	<code>\</code> (matrix left division)
Nonlinear equation of one variable	$f(a) = 0$	<code>fzero</code>
Nonlinear equations	$F(x) = 0$, n equations, n variables	<code>fsolve</code>

Least-Squares (Curve Fitting)

Type	Notation	Function
Linear least-squares	$\min_x \ C \cdot x - d\ _2^2, m \text{ equations, } n \text{ variables}$	\ (matrix left division)
Nonnegative linear-least-squares	$\min_x \ C \cdot x - d\ _2^2 \text{ such that } x \geq 0$	lsqnonneg
Constrained linear-least-squares	$\min_x \ C \cdot x - d\ _2^2 \text{ such that } A \cdot x \leq b, Aeq \cdot x = beq, l \leq x \leq u$	lsqlin
Nonlinear least-squares	$\min_x \frac{1}{2} \ F(x)\ _2^2 = \frac{1}{2} \sum_i F_i(x)^2 \text{ such that } l \leq x \leq u$	lsqnonlin
Nonlinear curve fitting	$\min_x \frac{1}{2} \ F(x, xdata) - ydata\ _2^2 \text{ such that } l \leq x \leq u$	lsqcurvefit